Ψηφιακά Συστήματα VLSI

5η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

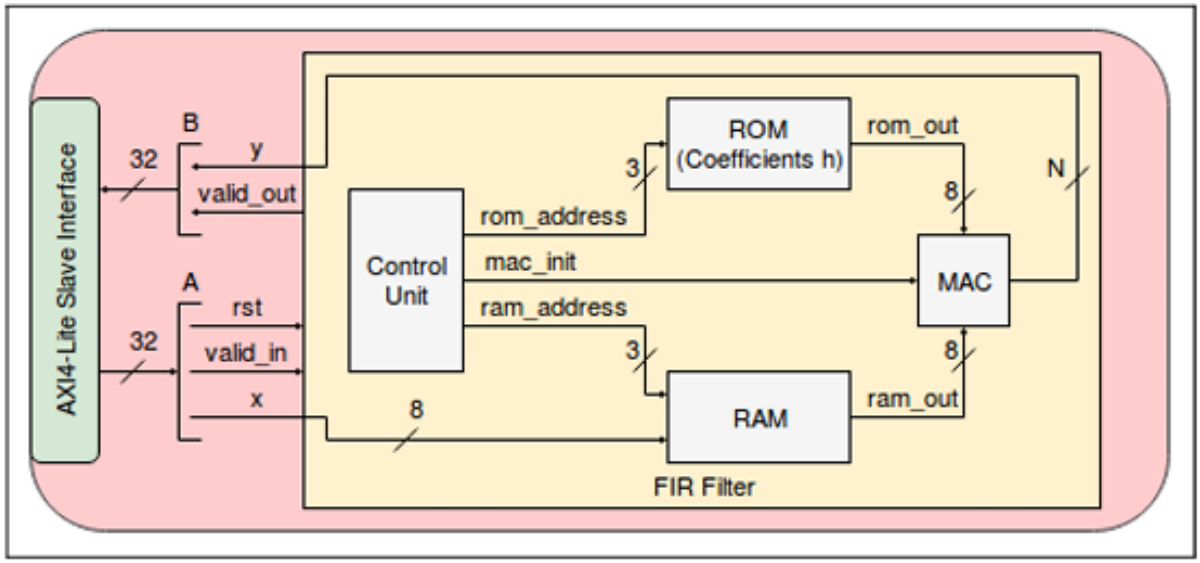
Δημήτριος Μιχαλακόπουλος (03120229)

Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Υλοποίηση FIR φίλτρου με AXI διεπαφή σε ZYNQ SoC FPGA

Στα πλαίσια αυτής της εργαστηριακής άσκησης μας ζητείται να υλοποιήσουμε ένα 8-tap FIR φίλτρο (M = 7), το οποίο θα υλοποιεί διεπαφή AXI4-Lite για την επικοινωνία με το ZYNQ Processing System (PS). Η αρχιτεκτονική που επιλέξαμε να χρησιμοποιήσουμε φαίνεται παρακάτω στην Εικόνα 1:



Εικόνα 1 : Αρχιτεκτονική FIR φίλτρου με διεπαφή AXI4-Lite.

```
A[7:0] = x
A[8] = valid_in
A[9] = rst
A[31:10] = not used
```

```
B[N-1:0] = y
B[N] = valid_out
B[31:N+1] = not used
```

Θα περιγράψουμε με σειρά τα βήματα που ακολουθήσαμε για να υλοποιήσουμε την εργαστηριακή άσκηση. Αρχικά, αφού ανοίξουμε New Project στο Vivado, πατάμε Tools → Create and Package New IP, για να δημιουργήσουμε ένα νέο IP για το FIR φίλτρο. Στο νέο παράθυρο που ανοίγει, προσθέτουμε ως design source τον VHDL κώδικα του 8-tap FIR φίλτρου που υλοποιήσαμε στην Εργαστηριακή Άσκηση 4. Έπειτα, σειρά έχει να τροποποιήσουμε το αρχείο υπεύθυνο για την διεπαφή AXI. Τα σημεία που τροποποιήσαμε φαίνονται παρακάτω:

```
121     ---- My Declarations
122     signal A, B : std_logic_vector(31 downto 0);
123     signal rst, valid_in, valid_out : std_logic;
124     signal x : std_logic_vector(7 downto 0);
125     signal y : std_logic_vector(18 downto 0);
126
127     component fir is
128         port (clk, rst, valid_in : in std_logic;
129               x : in std_logic_vector(7 downto 0);
130               valid_out : out std_logic;
131               y : out std_logic_vector(18 downto 0)
132         );
133     end component;
134
```

Σε αυτό το σημείο ορίζουμε τα σήματα που θα χρησιμοποιήσουμε καθώς και το φίλτρο FIR ως component.

```
402     -- Add user logic here
403     FIR0: fir port map (
404         clk => S_AXI_ACLK,
405         rst => rst,
406         x => x,
407         valid_in => valid_in,
408         y => y,
409         valid_out => valid_out
410     );
411
```

Σε αυτό το σημείο δημιουργούμε ένα instance του φίλτρου και αντιστοιχίζουμε τις θύρες του με τα κατάλληλα σήματα (port mapping).

```
412      process (S_AXI_ACLK) is
413          begin
414              A <= slv_reg0;
415              x <= A(7 downto 0);
416              valid_in <= A(8);
417              rst <= A(9);
418              B <= "000000000000" & valid_out & y;
419          end process;
420      -- User logic ends
```

Σε αυτό το σημείο υλοποιούμε το σήμα εισόδου A(32 bit) και το σήμα εξόδου B(32 bit) του φίλτρου σύμφωνα με την αρχιτεκτονική της Εικόνας 1. Η επιλογή να γράφουμε στον καταχωρητή slv_reg0 έγινε τυχαία.

```
226      process (S_AXI_ACLK)
227      variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
228      begin
229        if rising_edge(S_AXI_ACLK) then
230          if S_AXI_ARESETN = '0' then
231            slv_reg0 <= (others => '0');
232            --slv_reg1 <= (others => '0');
233            slv_reg2 <= (others => '0');
234            slv_reg3 <= (others => '0');
235          else
236            loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
237            if (slv_reg_wren = '1') then
238              case loc_addr is
239                when b"00" =>
240                  for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
241                    if ( S_AXI_WSTRB(byte_index) = '1' ) then
242                      -- Respective byte enables are asserted as per write strobes
243                      -- slave registor 0
244                      slv_reg0(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
245                    end if;
246                  end loop;
247                --when b"01" =>
248                  --for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
249                    --if ( S_AXI_WSTRB(byte_index) = '1' ) then
250                      -- Respective byte enables are asserted as per write strobes
251                      -- slave registor 1
252                      --slv_reg1(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
253                    --end if;
254                  --end loop;
255                when b"10" =>
256                  for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
257                    if ( S_AXI_WSTRB(byte_index) = '1' ) then
258                      -- Respective byte enables are asserted as per write strobes
259                      -- slave registor 2
260                      slv_reg2(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
261                    end if;
262                  end loop;
263                when b"11" =>
264                  for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
265                    if ( S_AXI_WSTRB(byte_index) = '1' ) then
266                      -- Respective byte enables are asserted as per write strobes
267                      -- slave registor 3
268                      slv_reg3(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
269                    end if;
270                  end loop;
```

```
271             when others =>
272               slv_reg0 <= slv_reg0;
273               slv_reg1 <= slv_reg1;
274               slv_reg2 <= slv_reg2;
275               slv_reg3 <= slv_reg3;
276           end case;
277           else slv_reg0(8) <= '0'; -- valid_in = 0
278         end if;
279       end if;
280     end if;
281   end process;
```

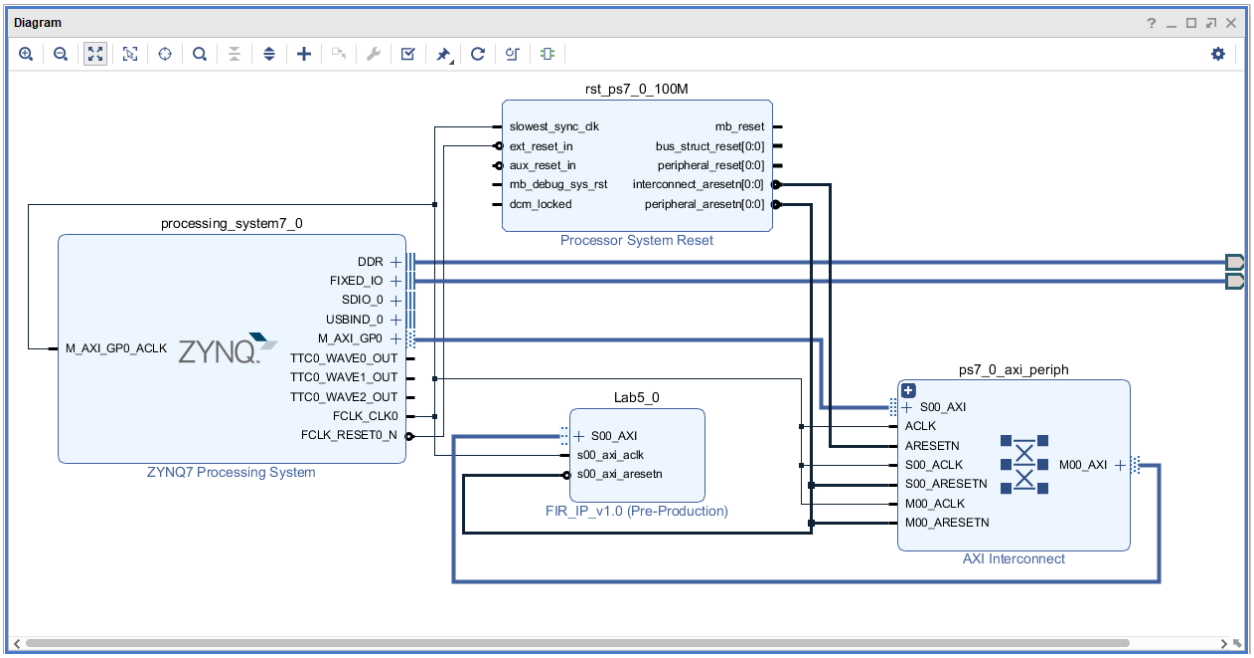
Σε αυτό το σημείο της διεπαφής γίνεται η διαδικασία write στους καταχωρητές, δηλαδή τα δεδομένα που στέλνουμε από το software γράφονται στο hardware. Εμείς, όμως, θέλουμε να χρησιμοποιήσουμε τον καταχωρητή slv_reg1 για να διαβάζουμε τα δεδομένα που προκύπτουν από την έξοδο του φίλτρου, επομένως σβήνουμε τον κώδικά του για να αποφύγουμε το Multiple Driver Error. Η επιλογή να διαβάζουμε από τον καταχωρητή slv_reg1 έγινε τυχαία. Επίσης, στην συνθήκη if (slv_reg_wren = '1'), που ελέγχει αν ενεργοποιείται η διαδικασία εγγραφής των καταχωρητών, βάζουμε μια συνθήκη else που κάνει το valid_in=0 ώστε κάθε έγκυρη είσοδος που στέλνουμε από το software να γράφεται μόνο μία φορά.

```
383     -- Output register or memory read data
384     process( S_AXI_ACLK ) is
385     begin
386       if (rising_edge (S_AXI_ACLK)) then
387         if ( S_AXI_ARESETN = '0' ) then
388           axi_rdata  <= (others => '0');
389         else
390         if valid_out = '1' then
391             slv_reg1 <= B;
392         elsif slv_reg_rden = '1' then
393             axi_rdata <= reg_data_out;
394             slv_reg1(19) <= '0';
395         else
396             slv_reg1 <= slv_reg1;
397           end if;
398         end if;
399       end if;
400     end process;
401
```

Σε αυτό το σημείο της διεπαφής γίνεται η διαδικασία read του καταχωρητή 1, δηλαδή τα δεδομένα που στέλνουμε από το hardware διαβάζονται στο software. Αν έχουμε valid_out=1 τότε η έξοδος του φίλτρου αποθηκεύεται στον slv_reg1 και το hardware περιμένει μέχρι να λάβει σήμα ότι το software είναι έτοιμο να

διαβάσει. Όταν λάβει σήμα slv_reg_rden='1', που σημαίνει ότι ενεργοποιείται η διαδικασία ανάγνωσης του καταχωρητή, τα δεδομένα του slv_reg1 περνάνε μέσω του axi_rdata στο software και μηδενίζεται το valid_out ώστε κάθε έγκυρη έξοδος που στέλνουμε από το hardware να διαβάζεται μόνο μία φορά.

Έχοντας τελειώσει με την διεπαφή AXI κάνουμε Package την IP μας και στο αρχικό Project κάνουμε Create Block Design. Εκεί εισάγουμε την IP του ZYNQ7 PS και του FIR φίλτρου και τρέχουμε Run Block Automation και Run Connection Automation. Το Block Design που προκύπτει φαίνεται παρακάτω:



Στην συνέχεια κάνουμε δεξί κλικ στο design μας και πατάμε Create HDL Wrapper. Έπειτα τρέχουμε RTL Analysis και Run Implementation για να ελέγξουμε αν θα προκύψουν errors ή σοβαρά warnings κατά την υλοποίηση. Τέλος, πατάμε το Generate Bitstream και αφού τελειώσει επιτυχώς αποθηκεύουμε το hardware file πατώντας File → Export → Export Hardware (Include bitstream).

Αφού τα κάνουμε όλα αυτά, πατάμε File → Launch SDK και ανοίγουμε το SDK για να γράψουμε ένα αρχείο σε γλώσσα C, το οποίο θα στέλνει τα δεδομένα εισόδου στο hardware και θα διαβάζει τα αντίστοιχα δεδομένα εξόδου που επιστρέφει το FIR φίλτρο. Ο κώδικας που υλοποιήσαμε φαίνεται παρακάτω:

```c
#include <stdio.h>
#include "platform.h"
#include "xparameters.h"
#include "sleep.h"
#include <inttypes.h>
#include "xil_types.h"
#include "xil_io.h"
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "sleep.h"
#include "S00_AXI.h"
#define IP_ADDR 0x43C00000

int main() {
    init_platform();
    unsigned int A, B, x, y, clk, rst, valid_in, valid_out;

    while (1) {
        xil_printf("Give me x: \n");
        scanf("%u", &x);
        xil_printf("Give me valid_in: \n");
        scanf("%u", &valid_in);
        xil_printf("Give me rst: \n");
        scanf("%u", &rst);
        A = rst << 9 | valid_in << 8 | x;
        xil_printf("A is : %u \n", A);
        S00_AXI_mWriteReg(XPAR_S00_AXI_0_S00_AXI_BASEADDR, S00_AXI_S00_AXI_SLV_REG0_OFFSET, A);
        B = S00_AXI_mReadReg(XPAR_S00_AXI_0_S00_AXI_BASEADDR, S00_AXI_S00_AXI_SLV_REG1_OFFSET);
        valid_out = B & 0x80000;
        valid_out = valid_out >> 19;
        y = B & 0x7FFFF;
        xil_printf("valid_out is: %u, y is: %u, B is %u\n\r", valid_out, y, B);

        if ((rst == 1) && (valid_in == 1)) {
            while (valid_out == 0) {
                B = Xil_In32(IP_ADDR + 0x04);
                valid_out = B & 0x80000;
                valid_out = valid_out >> 19;
            }
        }
        y = B & 0x7FFFF;
        xil_printf("valid_out is: %u, y is: %u, B is %u\n\r", valid_out, y, B);
    }
    cleanup_platform();
    return 0;
}

```

Αρχικά, πηγαίνουμε στο αρχείο xparameters.h και βρίσκουμε την διεύθυνση της IP μας την οποία ορίζουμε ως IP_ADDR. Έπειτα, ορίζουμε όλες τις απαραίτητες μεταβλητές, ζητάμε από τον χρήστη να δώσει τιμές για τις μεταβλητές εισόδου και σχηματίζουμε με bitwise left shift και bitwise or την μεταβλητή του σήματος εισόδου A. Με την συνάρτηση S00_AXI_mWriteReg γράφουμε την τιμή της

μεταβλητής A στον καταχωρητή slv_reg0 και αντίστοιχα με την συνάρτηση S00_AXI_mReadReg διαβάζουμε τον καταχωρητή slv_reg1 και αποθηκεύουμε το περιεχόμενό του στην μεταβλητή B. Στην συνέχεια με bitwise and και bitwise right shift απομονώνουμε τις τιμές valid_out και y του φίλτρου και τις τυπώνουμε στο terminal. Αν είχαμε έγκυρη είσοδο (valid_in=1) και το reset είναι ανενεργό (rst=0), τότε με την συνάρτηση Xil_In32 διαβάζουμε συνεχώς τον slv_reg1 που βρίσκεται στην διεύθυνση IP_ADDR + 4 και απομονώνουμε την τιμή του valid_out μέχρι αυτή να γίνεται ίση με 1, δηλαδή μέχρι το FIR φίλτρο να τελειώσει τους υπολογισμούς και να δώσει έγκυρη έξοδο, την οποία απομονώνουμε και με την σειρά της τυπώνουμε στο terminal. Σε κάθε άλλη περίπτωση το πρόγραμμα ζητάει από τον χρήστη να εισάγει νέες τιμές εισόδου. Τα σημεία που τυπώνουμε της τιμές των μεταβλητών A και B γίνονται για δική μας διευκόλυνση ώστε να μπορούμε να ελέγχουμε για τυχών λάθη κατά την είσοδο ή την έξοδο.